

Backend Take-Home Challenge

Asynchrone Aufgabenwarteschlange

Überblick

Entwirf und implementiere einen asynchronen Job-Processing-Service. Das System besteht aus einer REST-API, über die Jobs eingestellt und abgefragt werden, und mindestens einem Worker-Prozess, der die Jobs aus einer Queue entnimmt und verarbeitet.

Die Wahl des Tech-Stacks ist dir überlassen (Sprache, Framework, Datenbank, Queue-Mechanismus). Bitte begründe deine Entscheidungen im README.

Erwarteter Zeitaufwand: 5–8 Stunden. Wir bewerten Qualität, nicht Quantität — eine saubere, durchdachte Teillösung ist wertvoller als eine vollständige, aber unstrukturierte. Wenn du nicht alles umsetzt, ist das kein Ausschlusskriterium: Ein priorisierter Abschnitt „Offene Punkte“ im README (was würdest du als Nächstes angehen, und warum?) ist eine vollwertige Abgabe.

KI-Nutzung

Der Einsatz von KI-Assistenten (z. B. GitHub Copilot, Claude, ChatGPT) ist ausdrücklich erlaubt — solche Werkzeuge gehören auch bei uns zum Arbeitsalltag. Wir erwarten lediglich Transparenz: Dokumentiere im README, welche Tools du wofür eingesetzt hast und an welchen Stellen du generierte Vorschläge angepasst oder verworfen hast. Kein KI-Einsatz ist ebenso völlig in Ordnung — dann genügt ein Satz.

Entscheidend ist, dass die Architektur- und Designentscheidungen deine eigenen sind und im README nachvollziehbar begründet werden.

Fachlicher Kontext

Stell dir vor, du baust den Kern eines Systems, das rechenintensive Aufgaben im Hintergrund verarbeitet — z. B. Bildkonvertierung, Reportgenerierung oder Datenimporte. Die eigentliche Verarbeitung muss nicht real implementiert werden; ein `sleep()` von wenigen Sekunden als Platzhalter genügt.

Funktionale Anforderungen

1. Job einreichen

`POST /jobs`

Nimmt einen neuen Job entgegen. Jeder Job hat mindestens:

- einen Typ (z. B. `image_resize`, `report_generation`, `data_import`)
- eine Payload (beliebiges JSON, abhängig vom Typ)

Die API gibt sofort eine Bestätigung mit einer eindeutigen Job-ID zurück. Der Job wird nicht synchron verarbeitet.

2. Job-Status abfragen

`GET /jobs/{id}`

Gibt den aktuellen Status und die Metadaten eines Jobs zurück. Ein Job durchläuft mindestens folgende Zustände:

PENDING → PROCESSING → COMPLETED | FAILED

Die Antwort enthält mindestens:

- Job-ID
- Typ und Payload
- aktuellen Status
- Zeitstempel (erstellt, gestartet, abgeschlossen)
- bei **COMPLETED**: ein Ergebnis (simuliert)
- bei **FAILED**: eine Fehlermeldung

3. Jobs auflisten

GET /jobs

Gibt eine Liste aller Jobs zurück. Unterstützt mindestens eine Filtermöglichkeit nach Status (z. B. ?status=PENDING). Vollständige Paginierung ist nicht gefordert — eine einfache Begrenzung der Ergebnismenge (z. B. ?limit=50) genügt.

4. Worker-Verarbeitung

Mindestens ein Worker-Prozess nimmt Jobs aus der Warteschlange und verarbeitet sie:

- Der Worker setzt den Status auf **PROCESSING**, simuliert die Arbeit (Pause von 2–5 Sekunden) und setzt den Status anschließend auf **COMPLETED** oder **FAILED**.
- Simuliere eine Fehlerquote: ca. jeder fünfte Job soll fehlschlagen.

Nicht-funktionale Anforderungen

- **Keine doppelte Verarbeitung:** Auch wenn mehrere Worker gleichzeitig laufen, darf ein Job nicht doppelt verarbeitet werden.
- **Persistenz:** Jobs überleben einen Neustart des Services. Eine In-Memory-Lösung allein ist nicht ausreichend.
- **Fehlerbehandlung:** Die API gibt sinnvolle HTTP-Statuscodes und Fehlermeldungen zurück (z. B. 404 bei unbekannter Job-ID, 400 bei ungültigem Input).
- **Out of Scope:** Authentifizierung und Autorisierung sind ausdrücklich nicht gefordert — investiere deine Zeit in die Kernlogik.

Bonus (optional)

Die folgenden Punkte sind nicht notwendig, zeigen aber zusätzliches Können. Wähle höchstens ein bis zwei aus, wenn du noch Zeit hast:

- **Retry-Logik:** Fehlgeschlagene Jobs werden automatisch bis zu N-mal wiederholt, bevor sie endgültig als **FAILED** gelten.
- **Recovery verwaister Jobs:** Stürzt ein Worker mitten in der Verarbeitung ab, bleibt der Job in **PROCESSING** hängen. Implementiere einen Mechanismus (z. B. Timeout oder Heartbeat), der solche Jobs erkennt und wieder zur Verarbeitung freigibt.

- **Idempotenz:** `POST /jobs` akzeptiert optional einen Idempotency-Key, sodass wiederholte Requests (etwa Client-Retries) keinen doppelten Job erzeugen.
- **Priorisierung:** Jobs können mit einer Priorität eingereicht werden. Höher priorisierte Jobs werden bevorzugt verarbeitet.
- **Callback/Webhook:** Bei Abschluss eines Jobs wird eine konfigurierbare URL benachrichtigt.
- **Abbruch:** `DELETE /jobs/{id}` bricht einen noch nicht gestarteten Job ab.
- **Metriken-Endpoint:** `GET /metrics` liefert einfache Statistiken (Anzahl Jobs pro Status, durchschnittliche Verarbeitungszeit).
- **Containerisierung:** Ein `docker-compose.yml`, das API, Worker und Datenbank zusammen startet.

Abgabe

Dein Repository sollte enthalten:

1. Lauffähigen Code

Der Service lässt sich mit wenigen Schritten starten. Idealerweise reicht ein `docker-compose up` oder eine klar beschriebene Anleitung.

2. Tests

Automatisierte Tests für die wesentlichen Abläufe. Wir erwarten keinen bestimmten Abdeckungsgrad — zeig uns, wie du testest und welche Fälle du für wichtig hältst.

3. README mit folgenden Abschnitten

- **Architekturentscheidungen:** Warum hast du diesen Stack, diese Datenbank, diesen Queue-Mechanismus gewählt? Welche Alternativen hast du erwogen?
- **Startanleitung:** Schritt für Schritt, wie man das System lokal zum Laufen bringt.
- **Designentscheidungen:** Wo hast du bewusst Kompromisse gemacht? Was würdest du in einer Produktionsumgebung anders lösen?
- **KI-Einsatz:** Welche KI-Tools hast du wofür genutzt, und wo hast du generierte Vorschläge angepasst oder verworfen? (Bei keinem Einsatz genügt ein Satz.)
- **Skalierung:** Wie würdest du das System anpassen, wenn einzelne Jobs 30 Minuten oder länger laufen? Welche neuen Probleme entstehen — etwa bei Worker-Ausfällen oder Deployments? (2–3 Absätze genügen, keine Implementierung nötig.)
- **Offene Punkte (falls zutreffend):** Was hast du bewusst weggelassen, was würdest du als Nächstes angehen — und warum?

Bewertungskriterien

Bereich	Was wir uns ansehen
Architektur	Ist die Trennung von API, Queue und Worker sauber? Ist das System erweiterbar?
Korrektheit	Funktionieren die Zustandsübergänge zuverlässig? Werden Race Conditions vermieden?

Bereich	Was wir uns ansehen
Code-Qualität	Ist der Code lesbar, konsistent strukturiert und sinnvoll aufgeteilt?
Fehlerbehandlung	Wie geht das System mit ungültigen Eingaben, fehlgeschlagenen Jobs und Grenzfällen um?
Tests	Sind die wichtigsten Pfade getestet? Sind die Tests aussagekräftig?
Dokumentation	Zeigt das README, dass du deine Entscheidungen reflektiert und begründet hast?

Hinweise

- Du musst nicht alle Bonus-Aufgaben lösen. Fokussiere dich auf eine saubere Kernlösung.
- Wir bewerten keine bestimmte Sprache oder Framework-Wahl — zeig uns, womit du dich sicher fühlst.
- Wenn du an einer Stelle eine bewusste Abkürzung nimmst (z. B. Polling statt Pub/Sub), dokumentiere das. Ein reflektierter Kompromiss ist kein Fehler, sondern eine Stärke.
- Bei Fragen zur Aufgabenstellung: Triff eine begründete Annahme und dokumentiere sie. Genau das erwarten wir auch im Arbeitsalltag.